Functional Description

Name: Ethernet/IP Function Blocks for CODESYS

Software Used: CODESYS V3 SP19 (user need V3 SP15 or later to use sample program/library)

Communication Protocol: Ethernet/IP

Hardware Compatibility :

- Applied Motion Drives with Ethernet/IP compatibility

Software Compatibility:

Applied Motion Ethernet/IP Function Block Drive Series Compatibility Chart

Function Block	ST	STF	TSM	SSDC	SV200	MDX
AMP_Input_Assembly	Y	Y	Y	Y	Y	Y
AMP_Status_Code	Y	Y	Y	Y	Y	Y
AMP_Alarm_Code	Y	Y	Y	Y	Y	Y
AMP_Motor_Enable	Y	Y	Y	Y	Y	Y
AMP_Motor_Disable	Y	Y	Y	Y	Y	Y
AMP_Absolute_Move	Y	Y	Y	Y	Y	Y
AMP_Relative_Move	Y	Y	Y	Y	Y	Y
AMP_Normal_Stop	Y	Y	Y	Y	Y	Y
AMP_Crash_Stop	Y	Y	Y	Y	Y	Y
AMP_Alarm_Reset	Y	Y	Y	Y	Y	Y
AMP_SCL_Command	Y	Y	Y	Y	Y	Y
AMP_Point_to_Point_IO	Y	Y	Y	Y	Y	Y
AMP_Point_to_Point_FC	Y	Y	Y	Y	Y	Y
AMP_Jog_Move	Y	Y	Y	Y	Y	Y
AMP_Update_Jog_Speed	Y	Y	Y	Y	Y	Y
AMP_Q_Segment_Execute	Y	Y	Y	Y	Y	Y
AMP_Hard_Stop_Homing	N	N	Y	Y	Y	Y

References:

- Host Command Reference Manual

1 Contents

1 INTRODUCTION

This application note will show how to set up CODESYS project that uses Function Blocks in Ladder Logic Diagram (LD) to control an Ethernet/IP-enabled drive from Applied Motion Products.

These Function Blocks are intended to demonstrate the features of the Applied Motion Products Ethernet/IP implementation. They are being distributed locked and protected and may be used as-is in your PLC program and contact AMP if any modification is required. This document outlines the supplied Function Blocks, and provides the steps needed to get the motor spinning.

The functional descriptions found in the appendix provide details about the input and output variables of each Function Block.

1.1 Additional Content

Within the zip file that contained this PDF, you can find the following other contents:

- A file named AMP_EthernetIP_Library_LD_revx.compiled-library (for ladder logic) and AMP_EthernetIP_Library_ST_revx.compiled-library (for structured text), which contains the library with AMP Function Block to be imported into your program.
- The file AMP_EthernetIP_Sample_Program_LD which is an example program with all the Function Blocks imported, and one pre-configured drive TSM23X3B-IP using Ladder Logic Diagram.
- The file AMP_EthernetIP_Sample_Program_ST which is an example program with all the Function Blocks imported, and one pre-configured drive TSM23X3B-IP using Structured Text (ST).
    - This demo file assumes a drive IP addresses of 10.10.10.10

1.2 Reference Documentation

This document only explains each Function Block's purpose and functionality. It is not intended to be fully inclusive of all the drive's features and functionality. For more information regarding drive commands, input/output data structures, acronyms, operands, etc. please refer to:

i. Host Command Reference, 920-0002 Rev. W, 1/17/2018 (https://www.applied-motion.com/sites/default/files/hardware-manuals/Host-Command-Reference_920-0002P.PDF)

ii. Application Note APPN0040: "Hard-Stop Homing Commands for StepSERVO and SV200 Series" (https://www.applied-motion.com/sites/default/files/APPN0040A_Hard-Stop_Homing_StepSERVO-SV200.pdf)

iii. AMP YouTube video channel link: Applied Motion Products, Inc. - YouTube

iv. Application Note APPN0024: "Add-On Instructions for Allen Bradley® RSLogix5000® Software" (https://www.applied-motion.com/sites/default/files/APPN0024_FBs-for-RSLogix5000.zip)

v. Specific drive hardware manuals. These can be found in the *Downloads* section of the drive Product Page on the Applied Motion Products website (http://www.applied-motion.com)

1.3 Example Hardware

This application note was developed and primarily tested on the following hardware:

- CODESYS software V3.5 SP19
- Applied Motion Products TSM23X3B-IP – IP address 10.10.10.10

For determining compatibility of individual commands and features used by the Function Blocks with other hardware from Applied Motion Products, always refer to the Host Command Reference manual.

2 SPECIFICATION OF FUNCTION BLOCKs

The majority of Function Blocks provided are similar to each other in terms of parameters and common functionality. In this section we therefore document the basic common features of the Function Blocks

- Preview of Function Block on a rung
- Common sequence of operations
- Exceptions and notes

The remainder of the section will describe Function Blocks in specific terms: of their general description and parameters. In the appendix, you can find a specification of each of the Function Blocks provided. For each Function Block we provide

- A short description
- The list of parameters
- A normal sequence of operations
- Some exceptions and notes, if applicable

2.1 General Function Block Design and Use

2.1.1 PLC code examples

Figure 1 and Figure 2 are previews of how the Function Block will appear to the PLC user.

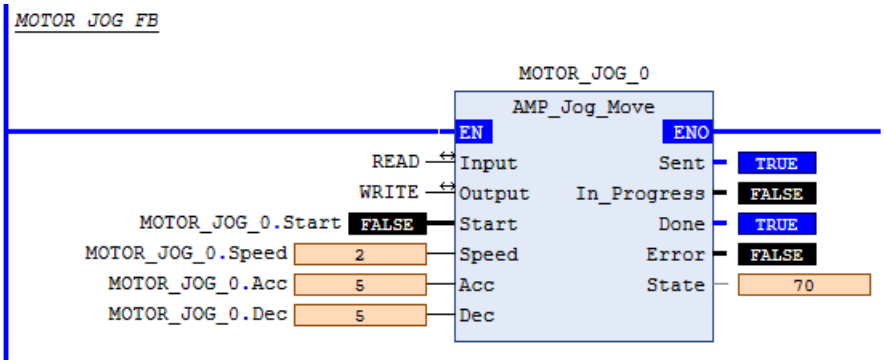


Figure 1: AMP_JOG_Move Function Block Instance Example

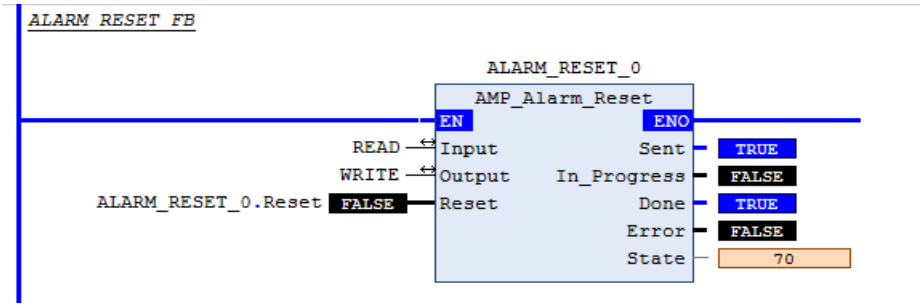


Figure 2: AMP_Alarm_Reset Function Block Instance Example

2.1.2 Typical Command Sequence

Many Function Blocks are simple wrappers for "native" drive commands (e.g. FL=Feed to Length, full details or which can be found in the Host Command Reference). Such Function Blocks have an appropriately named parameter used for triggering the start of the command sequence.

> *The built-in* EnableIn *parameter is only used to enable the execution of the* Function Block*, not to trigger the drive to perform an action.*

When the "trigger" input parameter transitions from false to true, the following sequence of steps occur:

- If the drive is not in the correct state (not enabled, faulted, etc), the Function Block sets its Error output and takes no further action.
- Otherwise, the Function Block outputs a command value of 0 (Idle) in case it was not already 0.
- Function Block ensures that the drive command word has been idle for a set time (130 milliseconds) or waits for that length of time before proceeding.
- Function Block copies or transforms Function Block inputs and outputs these to the drive (e.g. for a Relative Move it outputs the setpoint distance and the correctly scaled values of speed, acceleration, and deceleration parameters).
- Function Block outputs the command value corresponding to the Function Block basic function (e.g. a Relative Move corresponds to a "Feed to Length" (FL) command, i.e. setting the command word to a value of 0x8).
- Function Block waits at least a full I/O cycle (130 milliseconds) to ensure the drive saw the command, then resets the command word value to 0 (Idle) to prepare for the next command.
- Function Block sets its Sent output.

- The drive continually updates the status as the command progresses to completion (the completion condition varies per Function Block and is detailed within each individual specification further below).

Function Block waits for the drive status to indicate completion of the command (e.g. for a Relative Move this would be "In Position").

- Function Block sets its Done output.

2.1.3 Exceptions/Notes

2.1.4 Error output

If an error occurs during execution, the Function Block sets its Error output. The Error output is reset when the “trigger” parameter transitions from false to true.

2.1.5 Buffering and Sent output

Depending on the type of commands, the drive may sometimes buffer the command if it is not ready to execute it immediately. When using buffering, the Function Block’s Sent output can be integrated into the flow control logic to trigger several commands in rapid succession. Full details of Buffered vs Immediate type commands can be found in the Host Command Reference, and the Function Block descriptions in the appendix indicate whether the Function Blocks use Buffered or Immediate commands.

2.1.6 Using Done output with buffering

When using buffering, the Done output may not reflect intermediate completion states between successive commands. If intermediate completion checkpoints are desired, the user will need to develop custom completion logic.

2.1.7 Trigger signal is edge triggered.

When the “trigger” signal parameter transitions from true to false, the command is not stopped, but rather proceeds normally.

2.1.8 Abortion of command

Since the “trigger” signal parameter is rising edge triggered and cannot be used to abort a command, the user will need to use other means to stop/abort. For simple commands that involve moves, this can be done by executing the AMP_Normal_Stop or the AMP_Crash_Stop Function Block.

2.1.9 Using Done output with Interruption/Stopping

The interruption of a command using “Stop” Function Blocks will render invalid the Done output of the original Function Block command. To prevent confusion by the user, Function Block documentation in the appendix is clear about the exact meaning of the Done output for each given Function Block, and whether a command is subject to buffering or not.

3 EXAMPLE PROJECT

3.1 New project setup in CODESYS®

This section features a step-by-step procedure for setting up a new project that can communicate to an Applied Motion Ethernet/IP drive and use a selection of Function Blocks (Function Blocks) to handle common tasks. The procedure can also be used to add a new drive and Function Blocks to an existing project. Note that a demo project pre-loaded with all the Function Blocks is also included in the ZIP folder with this application note: - AMP_EthernetIP_Sample_Program_LD

3.2 STEP 1 Create a new standard project

Open CODESYS software and create a new standard CODESYS project (V 3.5 SP 19)

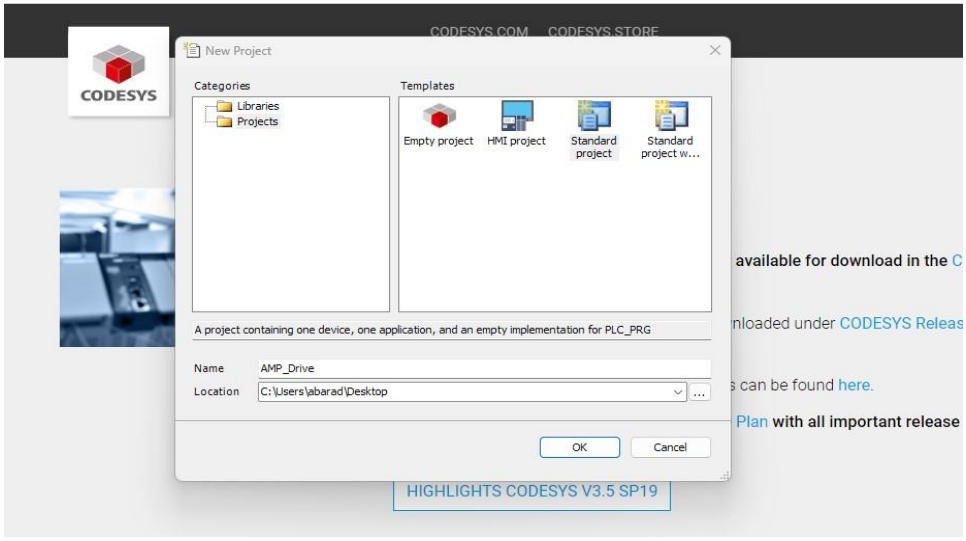


Figure 3: New Project Window in Codesys

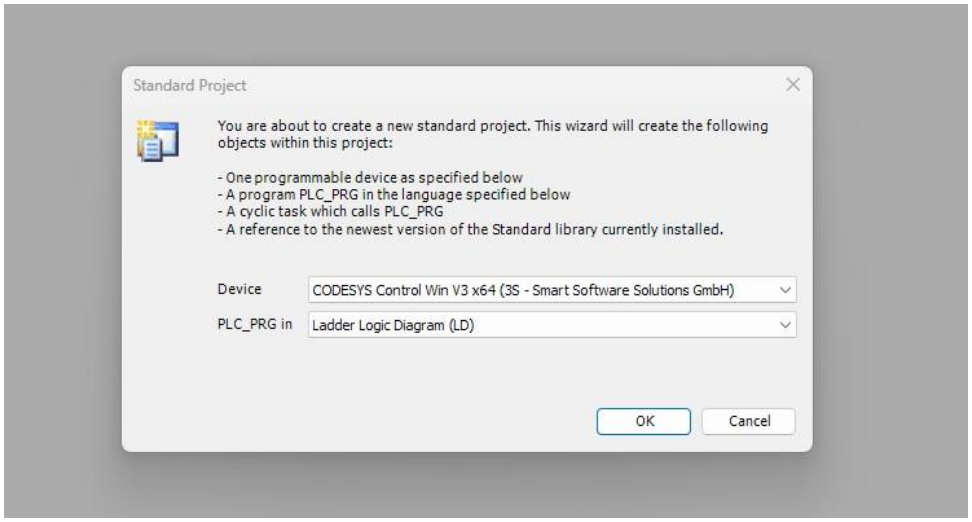


Figure 4: Device Selection and Main IEC 61131-3 Language Selection

3.3 STEP 2 I/O Configuration

3.3.1 EDS installation

Download the drive's eds file from AMP website and install it in CODESYS.
Tools > Device Repository

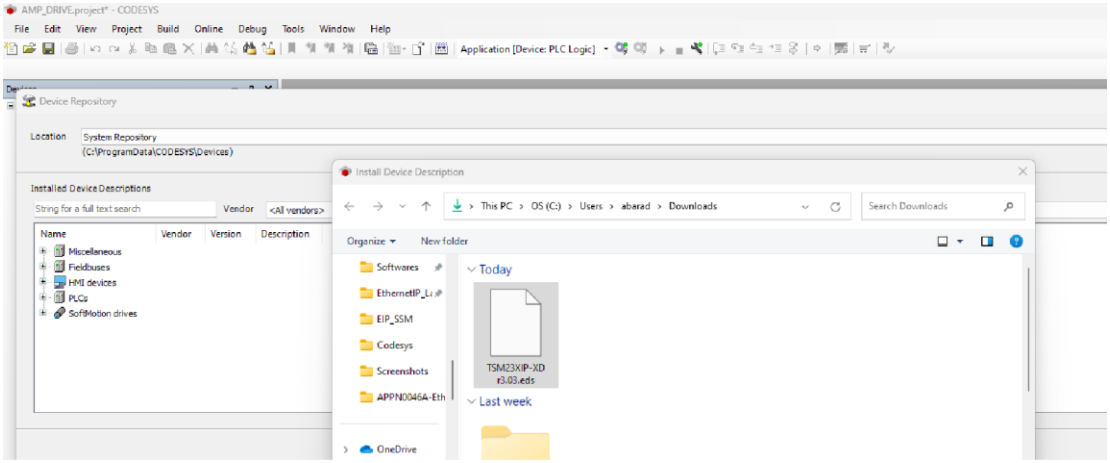


Figure 5: Applied Motion Files for installation in Codesys

3.3.2 Add an ethernet adapter

Right click on Device (CODESYS Control Win v3 x64) from Menu tree and add Ethernet Adapter and close the popup screen.

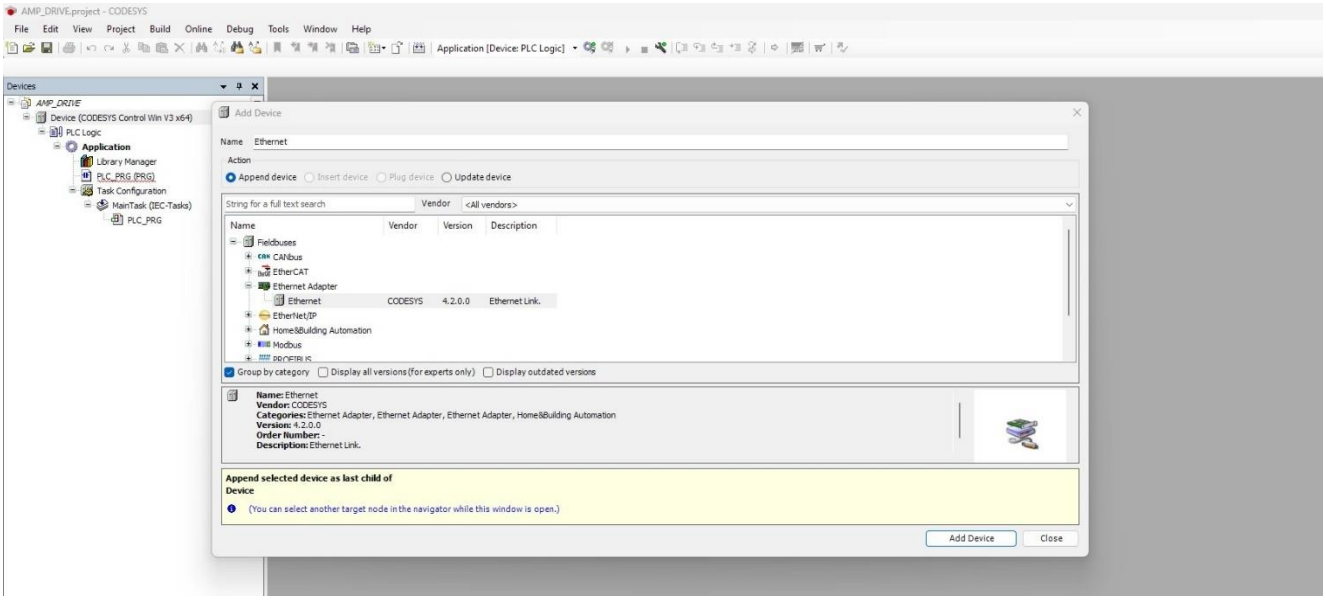


Figure 6: Add Device Pop Up Window

3.3.3 Add an ethernet scanner

Right click on added Ethernet (Ethernet) > Add Device > Fieldbuses > EtherNet/IP Scanner > EtherNet/IP Scanner and close the popup screen.

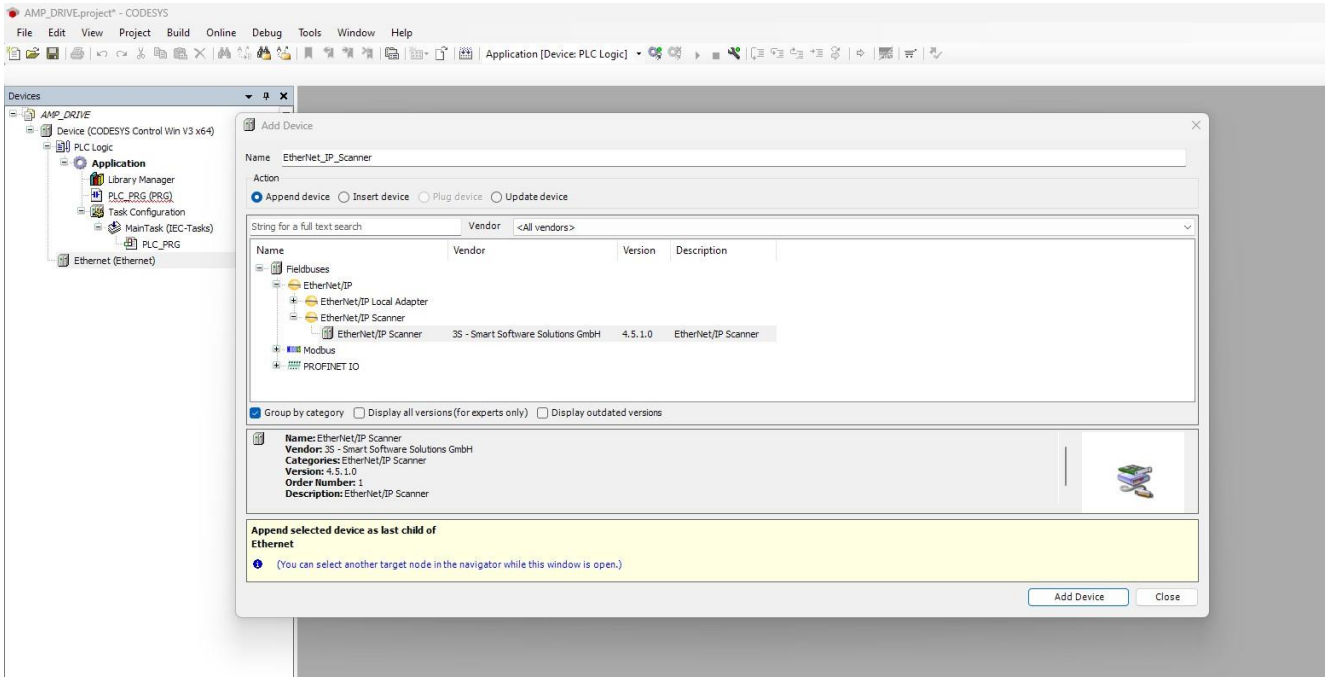


Figure 7: Ethernet/IP Scanner Selection in Add Device Workspace

3.3.4 Add an AMP Drive

Right click on added EtherNet_IP_Scanner (EthernetNet_IP_Scanner) > Add Device > Fieldbuses > EtherNet/IP Remote Adapter > Select available AMP drive and close the popup screen.

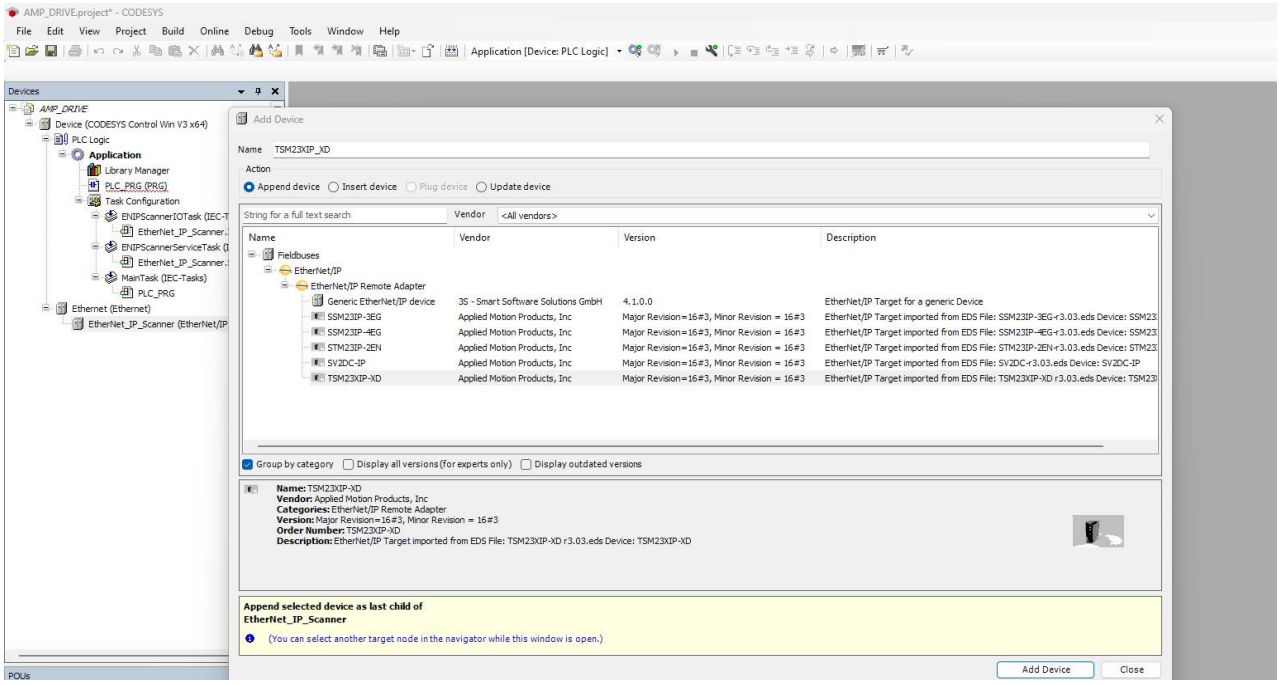


Figure 8: Applied Motion Ethernet/IP Device Selection

3.4 STEP 3 Configure the Ethernet Hardware Module

Double click on AMP Drive, Open the general tab, fill in the IP address of the AMP drive and uncheck the check match.

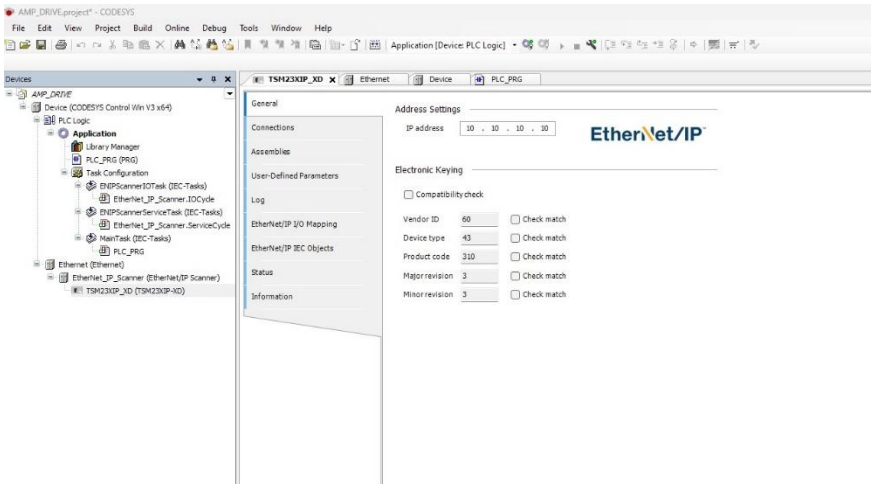


Figure 9: IP Address Configuration in New Device Workspace

Define READ Array[0..13] OF DINT and WRITE Array[0..15] OF DINT variables (user defined) and map them into the drive input and output assemblies as shown below

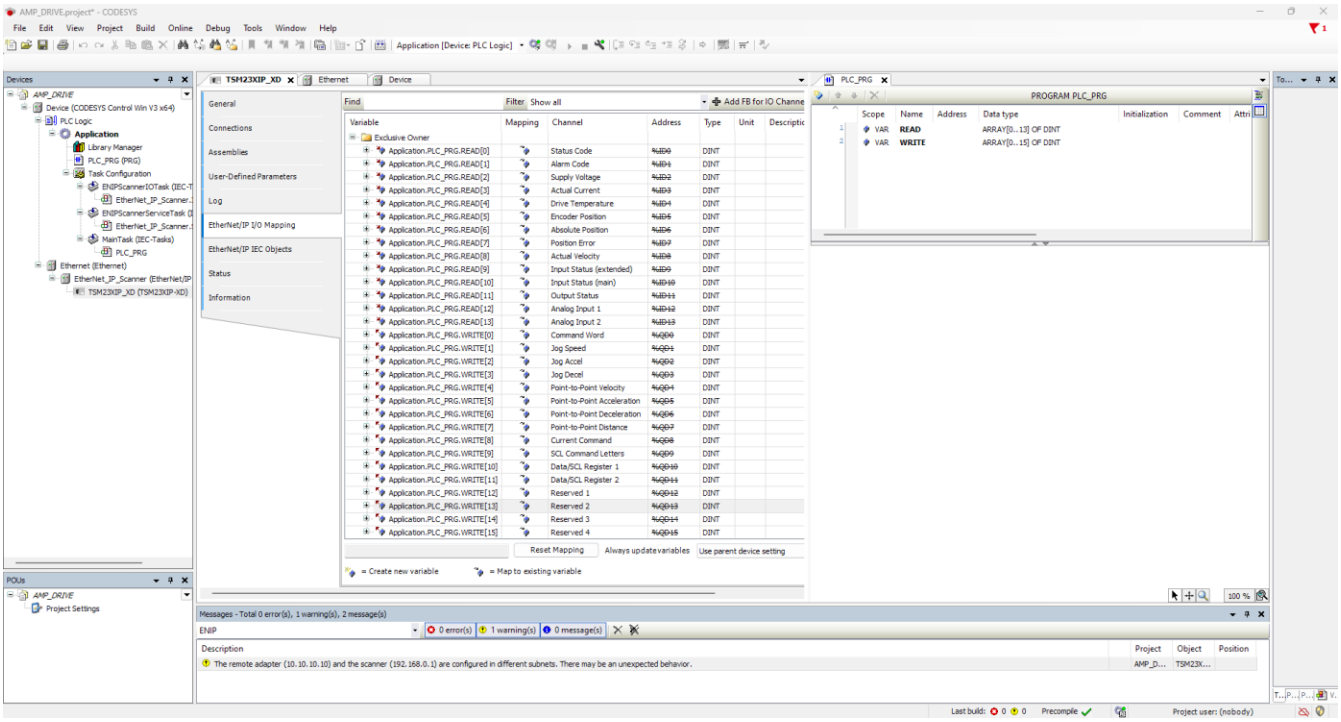
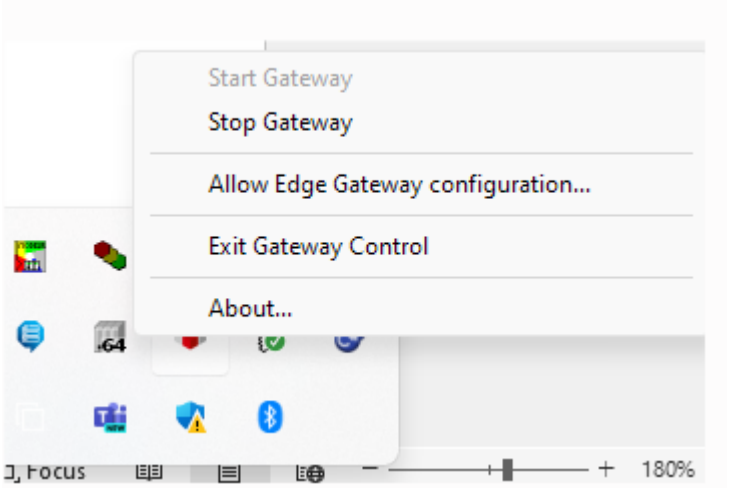


Figure 10: I/O Mapping of Ethernet/IP Variables

3.5 STEP 4 Gateway setup and Ethernet selection

3.5.1 After adding the drive, turn on CODESYS Gateway from windows Taskbar if it is not on.



3.5.2 Activate gateway connection:

Add your PC name/Controller name (Click on Scan network if needed) on Device > Communication Settings window, after that select available ethernet adapter as the same range of the drive.

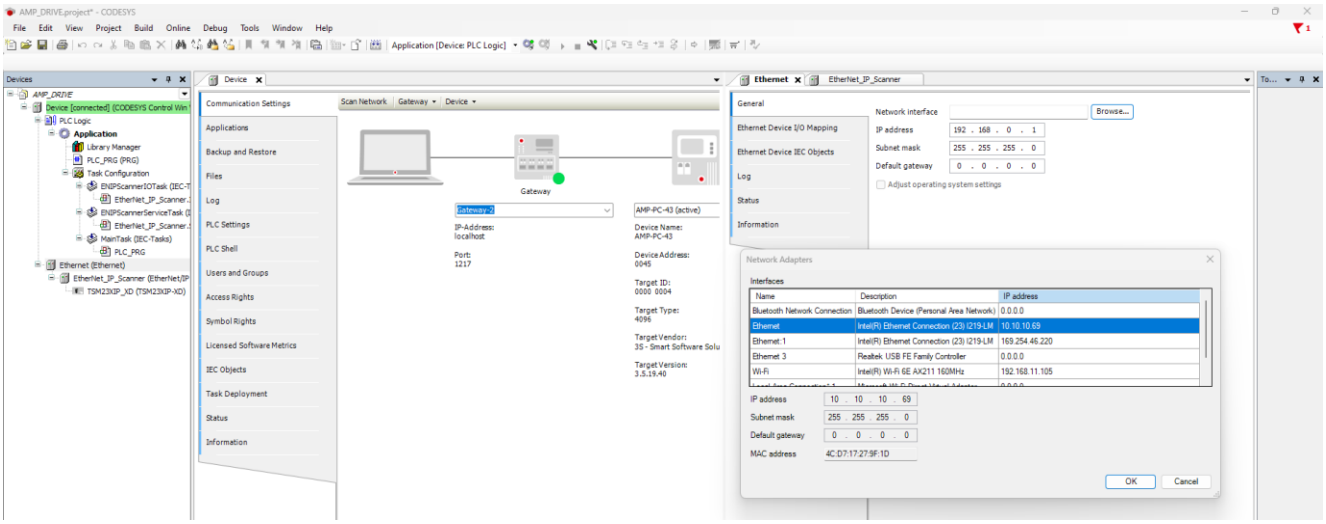


Figure 11: CODESYS Gateway setup

3.6 STEP 5 Confirm drive is connected

After connecting to the gateway and logged into the CODESYS, start runtime and it should show as below: -

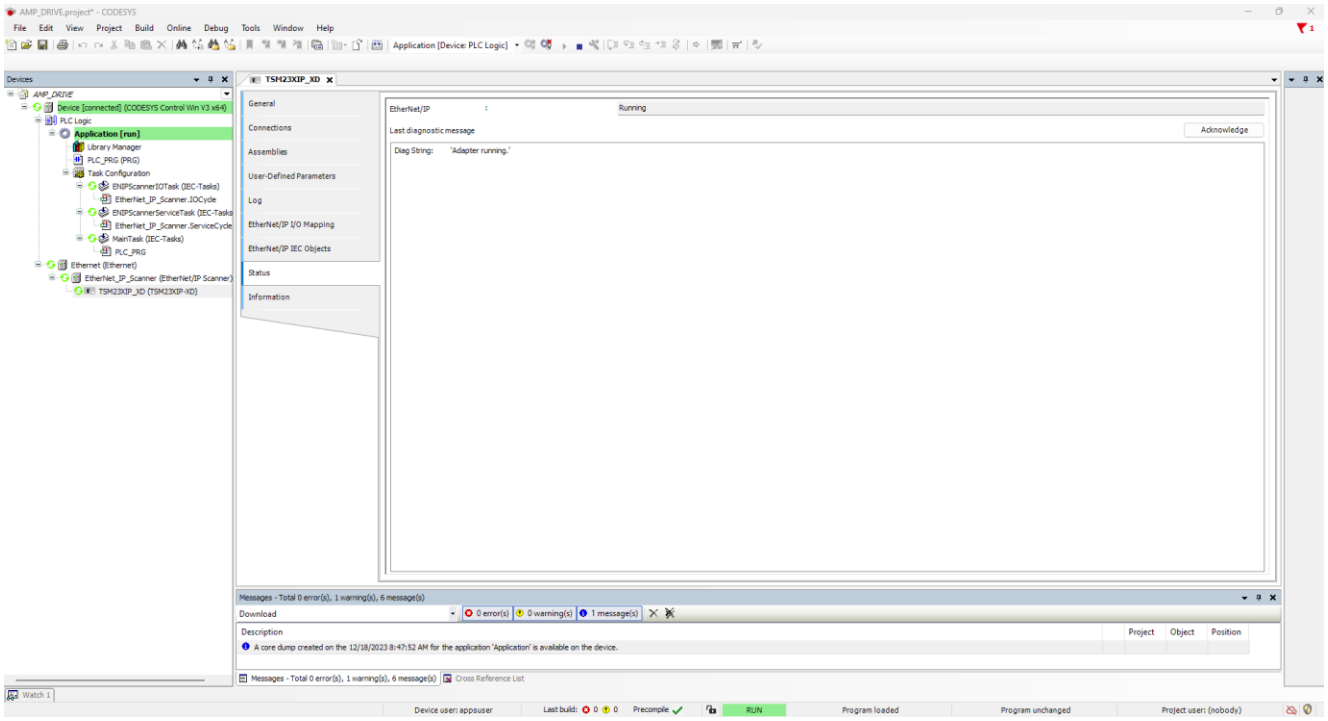


Figure 12: Codesys Environment when Gateway is Connected

3.7 STEP 6 Install Standard and AMP Library

3.7.1 You can find AMP Compiled library from .zip file from website.

Now install library from Library Manager > Library Repository > Install
(AMP library file is in the compiled format, one is for ladder logic and other one is for structured text)

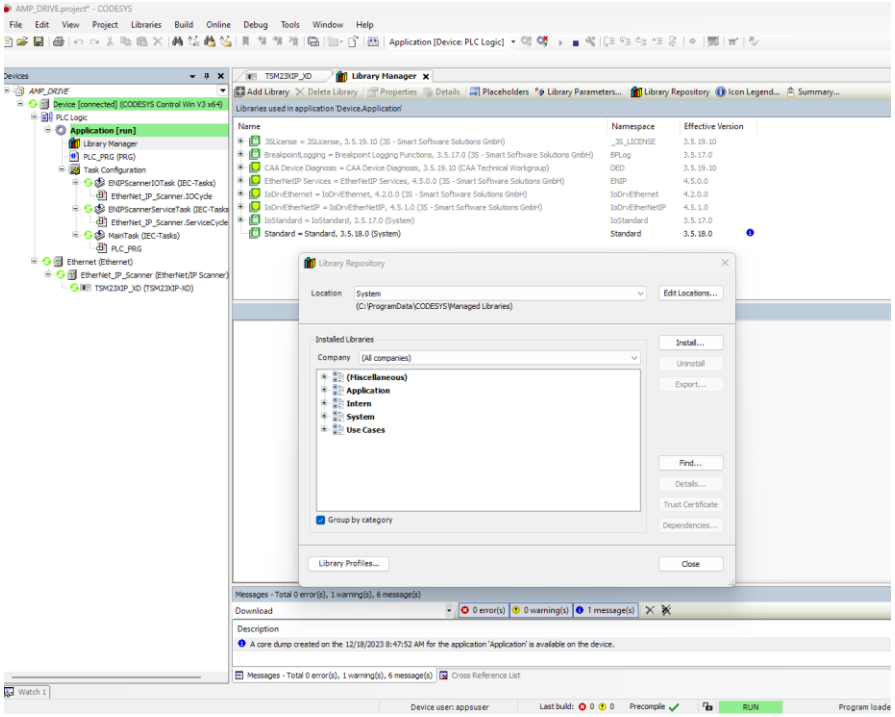


Figure 13: Library Manager Tool Workspace

3.7.2 Add AMP library

Add AMP library from Library Manager >Add Library > Miscellaneous > AMP_EthernetIP

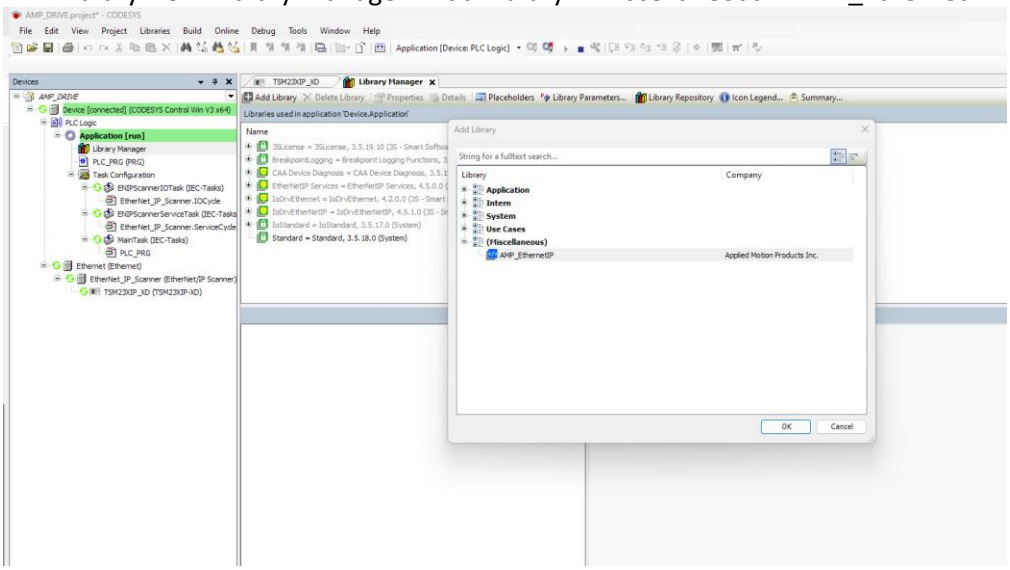


Figure 14: Add Library Workspace from Library Manager

3.7.3 Add the Standard library

If you do not have the standard library installed, install it from Tools > Library Manager > Add Library > Standard

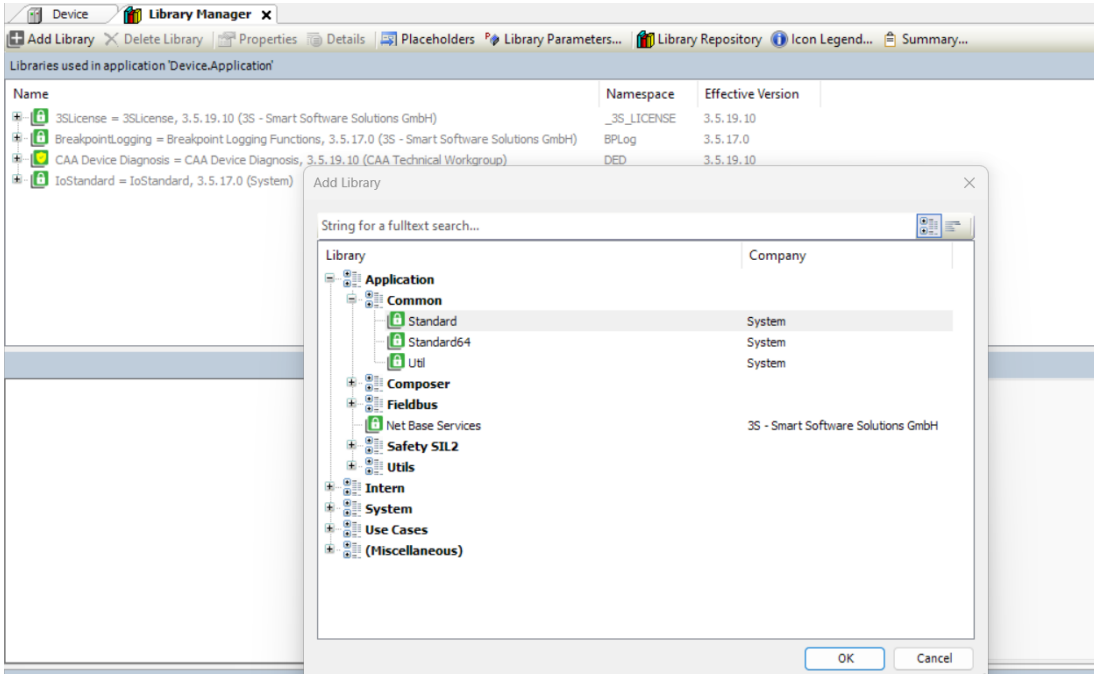


Figure 15: Standard Library Location during Add Library option in Library Manager

3.8 STEP 7 Confirm all the Imports

Once you have successfully installed and added libraries, you can see all the function blocks details as below: -

Note: Applied Motion Function Blocks are protected. If you need to modify any of the function blocks, please contact Applied Motion.

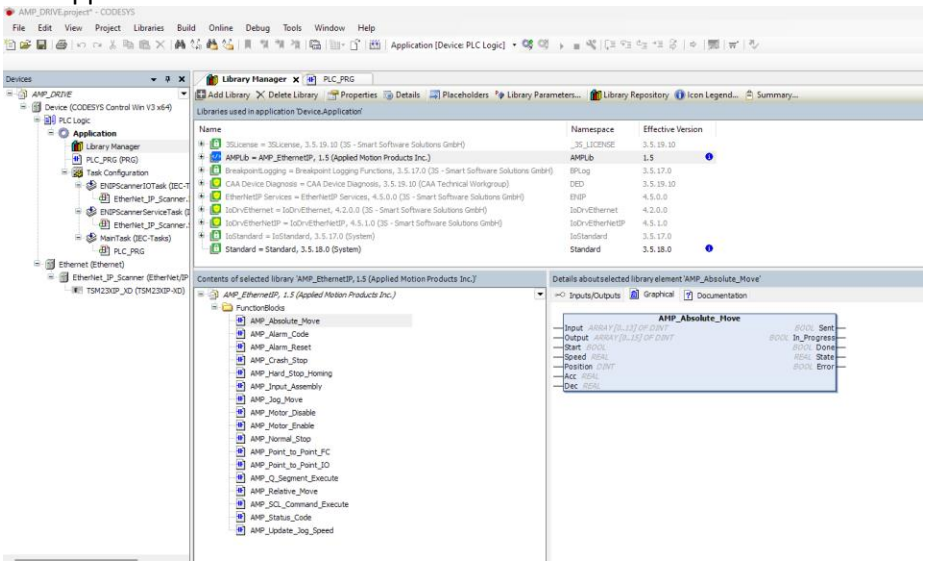


Figure 16: Applied Motion Library in Library Mana

3.9 STEP 8 Place a Function Block

3.9.1 Function Block creation

In the PLC_PRG primary function block, insert a [Box with EN/ENO] from the Toolbox. Double click on the type of function block to assign the box with one of the Applied Motion function blocks from the installed library. Once the Function Block has been assigned, the READ and WRITE variables (User Defined) can be assigned from the list.

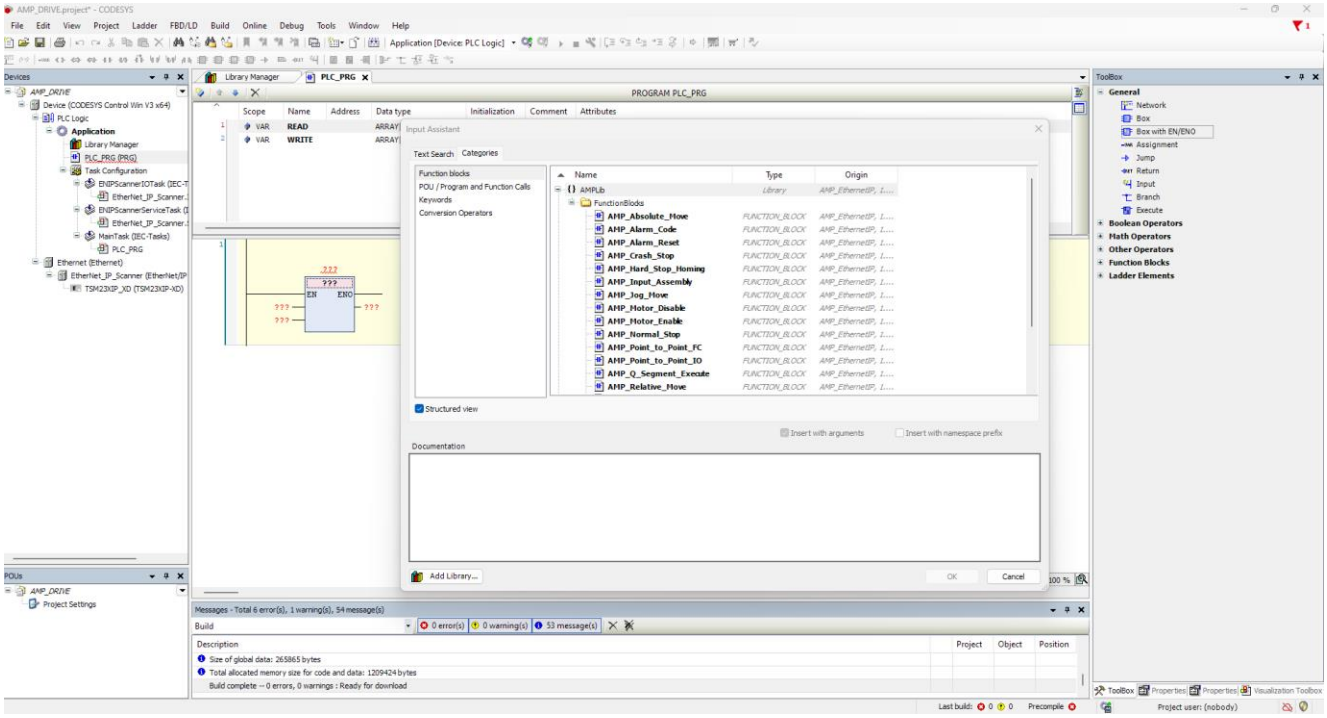


Figure 17: Input Assistant for assigning Variables to a Function Block

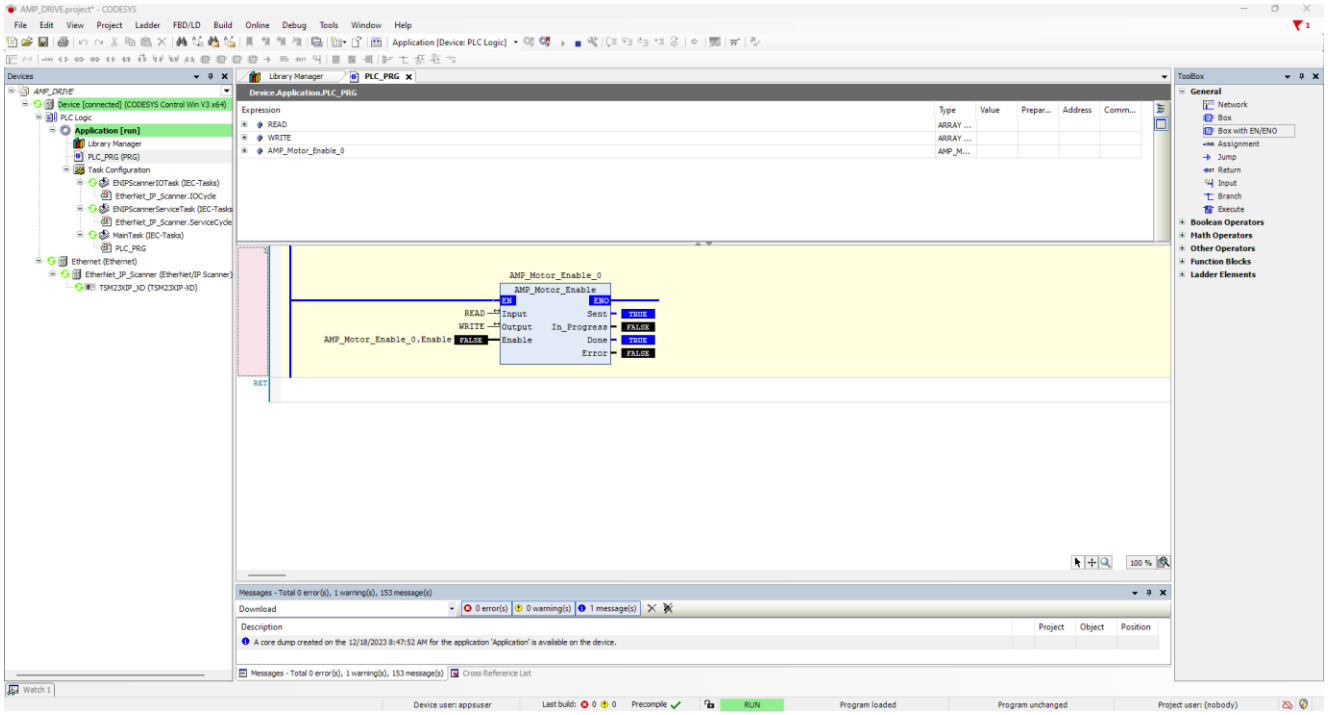


Figure 18: Example Function Block setup for Applied Motion Function Blocks

3.10 STEP 9 Add More FUNCTION BLOCKs

3.10.1.1 Repeat previous steps and add more Function Blocks and assign Input and output.

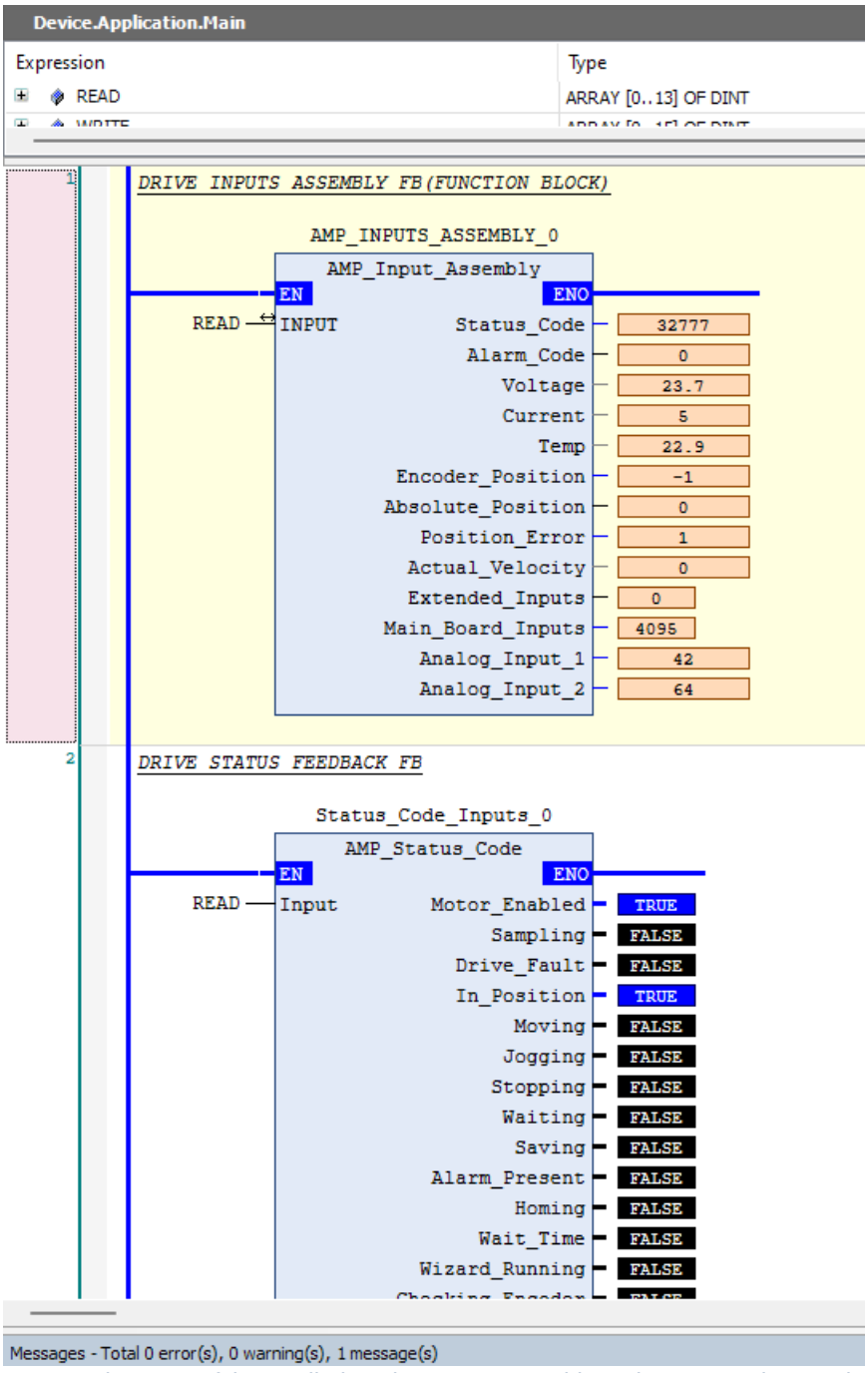


Figure 19: Example Setup of the Applied Motion Input Assembly and Status Code Function Blocks

4 APPENDIX: FUNCTION BLOCKS

Following is a comprehensive list of all Function Blocks included in the ZIP folder accompanying this Application Note:

4.1 Applied Motion Ethernet/IP Function Block Compatibility by Drive Series

Function Block	ST	STF	TSM	SSDC	SV200	MDX
AMP_Input_Assembly	Y	Y	Y	Y	Y	Y
AMP_Status_Code	Y	Y	Y	Y	Y	Y
AMP_Alarm_Code	Y	Y	Y	Y	Y	Y
AMP_Motor_Enable	Y	Y	Y	Y	Y	Y
AMP_Motor_Disable	Y	Y	Y	Y	Y	Y
AMP_Absolute_Move	Y	Y	Y	Y	Y	Y
AMP_Relative_Move	Y	Y	Y	Y	Y	Y
AMP_Normal_Stop	Y	Y	Y	Y	Y	Y
AMP_Crash_Stop	Y	Y	Y	Y	Y	Y
AMP_Alarm_Reset	Y	Y	Y	Y	Y	Y
AMP_SCL_Command	Y	Y	Y	Y	Y	Y
AMP_Point_to_Point_IO	Y	Y	Y	Y	Y	Y
AMP_Point_to_Point_FC	Y	Y	Y	Y	Y	Y
AMP_Jog_Move	Y	Y	Y	Y	Y	Y
AMP_Update_Jog_Speed	Y	Y	Y	Y	Y	Y
AMP_Q_Segment_Execute	Y	Y	Y	Y	Y	Y
AMP_Hard_Stop_Homing	N	N	Y	Y	Y	Y

4.2 AMP_Input_Assembly Function Block

4.2.1 Overview

The Input Assembly Function Block takes raw input data from the drive (Input Assembly 0x65) and re-interprets the data, using either scaling or copying, to appropriately named outputs.

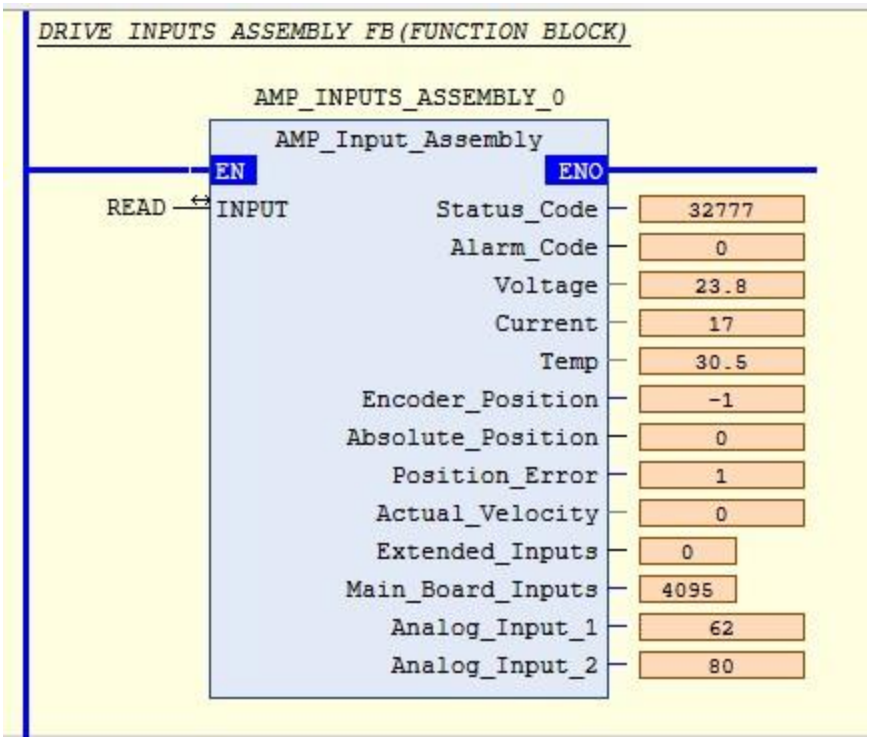


Figure 20: Function Block as Seen in Ladder Logic in Codesys V3.5 SP19

4.2.2 Functionality

The Input Assembly Function Block does not use the typical command sequence as it does not require sending any data to the drive. Here are the steps:

- The Function Block copies the raw data from the input assembly (Input parameter), in this example it is READ[0](First DINT of READ)
- The Function Block copies and/or scales data from the Input and outputs them to named fields.

4.2.3 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Status_Code	Output	DINT	Status Code
Alarm_Code	Output	DINT	Alarm Code
Voltage	Output	REAL	Supply Voltage (V)
Current	Output	REAL	Actual Current (mA)
Temp	Output	REAL	Drive Temperature (1.0°C)
Encoder_Position	Output	DINT	Encoder Position (20,000 counts = 1 shaft turn)
Absolute_Position	Output	DINT	Absolute Position (20,000 counts = 1 shaft turn)
Velocity	Output	REAL	Actual Velocity (rev/sec, 0 when no encoder)
Position Error	Output	REAL	Position error from the encoder
Entended_Inputs	Output	INT	Input Status (Extended)
Main_Board_Inputs	Output	INT	Input Status (Main board)
Analog_Input_1	Output	DINT	Analog Input 1 (ADC counts, 0=min V, 16383 = max V)
Analog_Input_2	Output	DINT	Analog Input 2 (ADC counts, 0=min V, 16383 = max V)

4.3 AMP_Status_Code Function Block

4.3.1 Overview

The Status Code Function Block uses the raw data stored in the Status Code word (Element 0, Ref 2, page 347) to set individual appropriately named Boolean outputs. The equivalent drive command is SC.

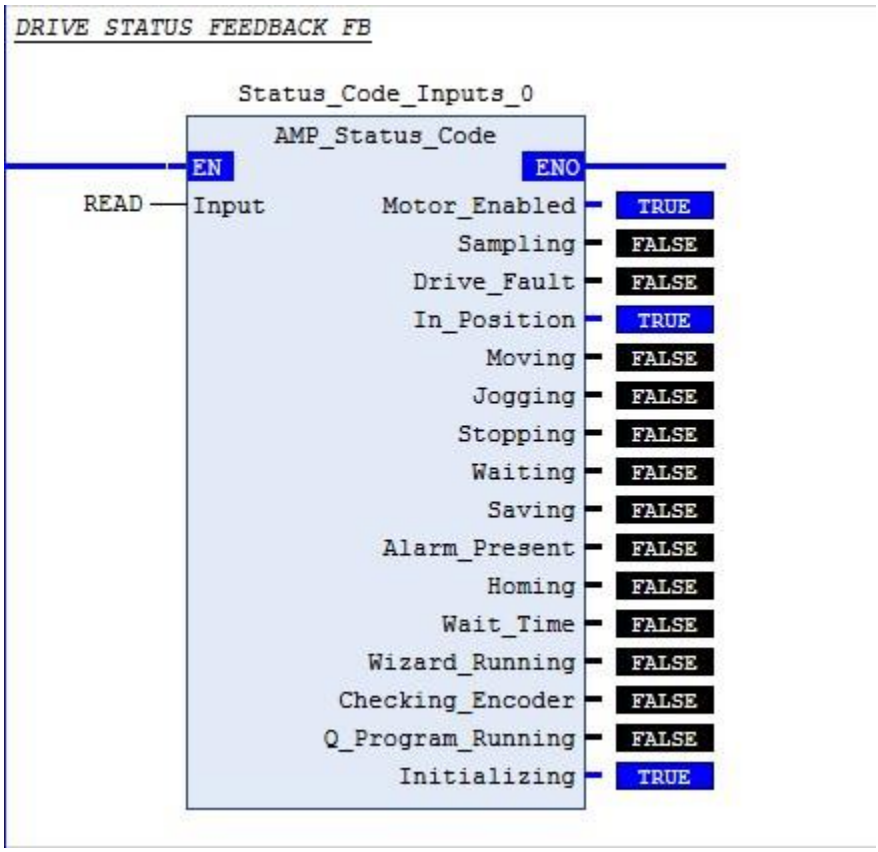


Figure 21: Applied Motion Status Code Function Block

4.3.2 Functionality

The Status Code Function Block does not use the typical command sequence as it does not require sending any data to the drive. Here are the steps:

- The Function Block copies individual Booleans from the Status Code word (Element 0 from Input Assembly 0x65) and sets appropriately named outputs.

4.3.3 Parameters

Name	Usage	Data Type	Description
Input	Input	ARRAY [0..13] OF DINT	Drive Status Code word (Element 0 from Input Assembly 0x65)
Enabled	Output	BOOL	Motor Enabled (Motor Disabled if this bit = 0)
Sampling	Output	BOOL	Sampling (for Quick Tuner)
Drive_Fault	Output	BOOL	Drive Fault (check Alarm Code)
In_Position	Output	BOOL	In Position (motor is in position)
Moving	Output	BOOL	Moving (motor is moving)
Jogging	Output	BOOL	Jogging (currently in jog mode)
Stopping	Output	BOOL	Stopping (in the process of stopping from a stop command)
Waiting	Output	BOOL	Waiting (for an input; executing a WI command)
Saving	Output	BOOL	Saving (parameter data is being saved)
Alarm_Present	Output	BOOL	Alarm present (check Alarm Code)
Homing	Output	BOOL	Homing (executing an SH command)
Wait_Time	Output	BOOL	Waiting (for time; executing a WD or WT command)
Wizard_Running	Output	BOOL	Wizard running (Timing Wizard is running)
Checking_Encoder	Output	BOOL	Checking encoder (Timing Wizard is running)
Q_Program_Running	Output	BOOL	Q Program is running
Initializing	Output	BOOL	Initializing (happens at power up) ; Servo Ready (for SV200 drives only)

4.4 AMP_Alarm_Code Function Block

The Alarm Code Function Block uses the raw data stored in the Alarm Code word (Element 1 from Input Assembly) to set individual appropriately named Boolean outputs. The equivalent drive command is AC.

4.4.1 Overview

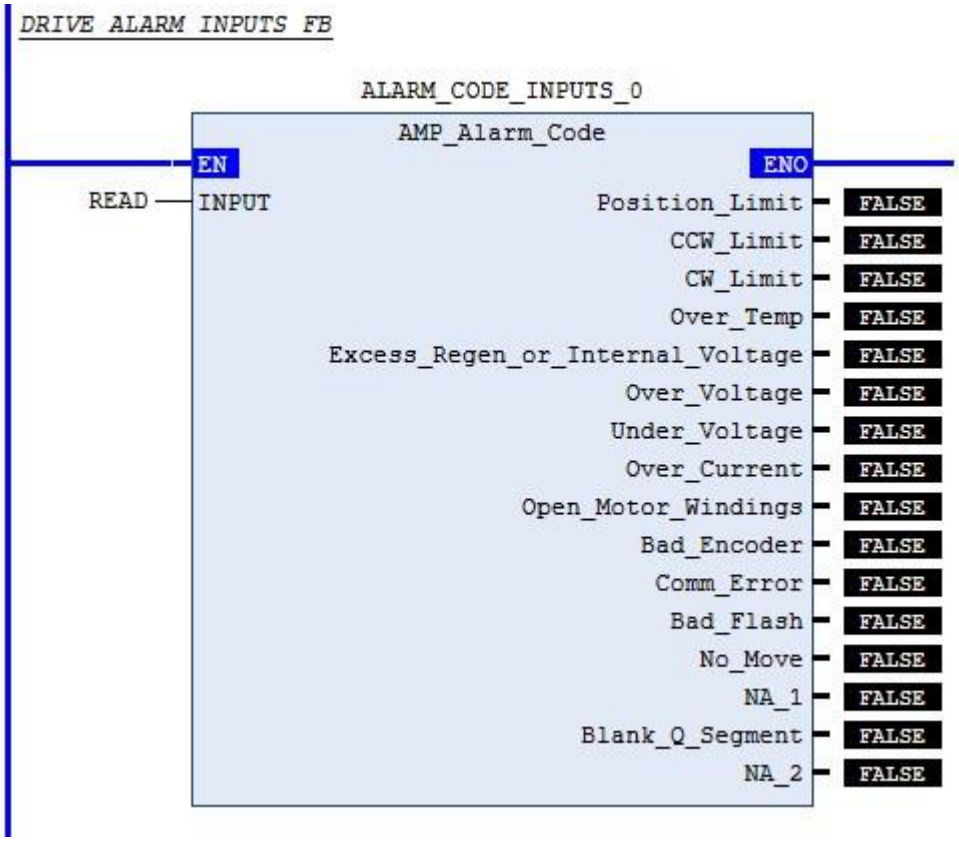


Figure 22: Applied Motion Alarm Code Function Block

4.4.2 Functionality

The Alarm Code Function Block does not use the typical command sequence as it does not require sending any data to the drive. Here are the steps:

- The Function Block copies individual Booleans from the Alarm Code word (Element 1 from Input Assembly, as seen in Reference Documents HCR) and sets appropriately named outputs.

4.4.3 Parameters

Name	Usage	Data Type	Description
Input	Input	ARRAY [0..13] OF DINT	Drive Alarm Code word (Element 1 from Input Assembly 0x65)
Position_Limit	Output	BOOL	Position Limit
CCW_Limit	Output	BOOL	CCW Limit
CW_Limit	Output	BOOL	CW Limit
Over_Temp	Output	BOOL	Over Temperature
Excess_Regen_or_Internal_Voltage	Output	BOOL	Excess Regen or Internal Voltage
Over_Voltage	Output	BOOL	Over Voltage
Under_Voltage	Output	BOOL	Under Voltage
Over_Current	Output	BOOL	Over Current
Open_Motor_Windings	Output	BOOL	Bad Hall Sensor / Open Motor Windings
Bad_Encoder	Output	BOOL	Bad Encoder
Comm_Error	Output	BOOL	Comm Error
Bad_Flash	Output	BOOL	Bad Flash
Wizard_Failed	Output	BOOL	Wizard Failed / No Move
Current_Foldback	Output	BOOL	Current Foldback / Motor Resistance
Blank_Q_Segment	Output	BOOL	Blank Q Segment
No_Move	Output	BOOL	No Move

4.5 AMP_Motor_Enable Function Block

4.5.1 Overview

The Motor Enable Function Block restores drive current to motor. The equivalent drive command is ME.

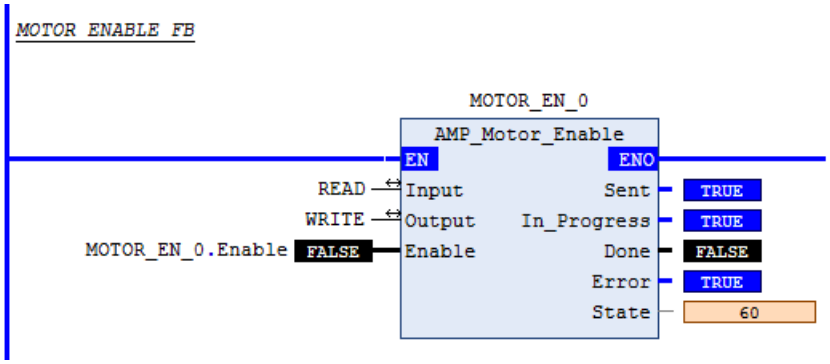


Figure 23: Applied Motion Motor Enable Function Block

4.5.2 Functionality

The Motor Enable Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: none
- Drive Command: "Motor Enable" (command word value of 0x2).
- Done output: when drive status indicates it is “Enabled”.

Type: Buffered

4.5.3 Parameters

Name	**Usage**	**Data Type**	**Description**
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Name	**Usage**	**Data Type**	**Description**
Enable	Input	BOOL	Enables the drive
Sent	Output	BOOL	Enable has been sent to drive
In_Progress	Output	BOOL	Enable in progress

Done	Output	BOOL	Enable completed without errors – Motor is Enabled
Error	Output	BOOL	Enable failed to complete (Always false – Future use only)

4.6 AMP_Motor_Disable Function Block

4.6.1 Overview

The Motor Disable Function Block disables motor outputs (reduces motor current to zero). The equivalent drive command is MD.

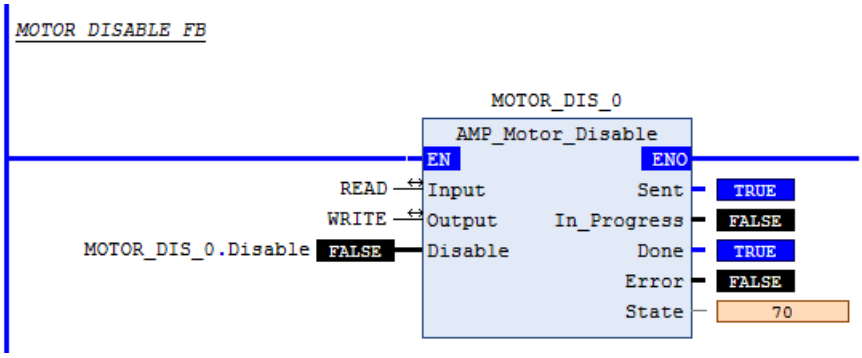


Figure 24: Applied Motion Motor Disable Function Block

4.6.2 Functionality

The Motor Disable Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: none
- Drive Command: "Motor Disable" (command word value of 0x4).
- Done output: when drive status indicates it is not “Enabled”.
- Type: Buffered

4.6.3 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Disable	Input	BOOL	Disables the drive
Sent	Output	BOOL	Disable has been sent to drive
In_Progress	Output	BOOL	Disable in progress
Done	Output	BOOL	Disable completed without errors – Motor is not Enabled

Error	Output	BOOL	Disable failed to complete (Always false – Future use only)

4.7 AMP_Absolute_Move Function Block

4.7.1 Overview

The Absolute Move Function Block is used to move an axis to a specified absolute position. The equivalent drive command is FP.

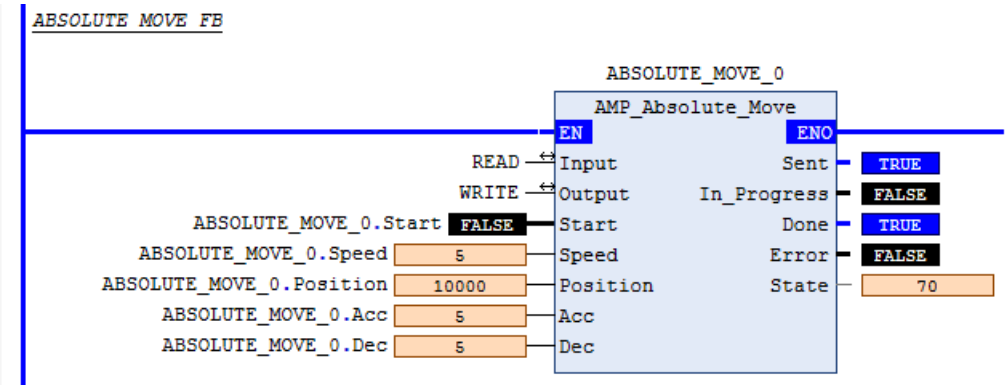


Figure 25: Applied Motion Absolute Move Function Block

4.7.2 Functionality

The Absolute Move uses a typical command sequence with the following specific differences:

- Parameters output to drive: Distance (=Target absolute position), as well as the correctly scaled values of parameters: Speed, Acc and Dec are sent via the output assembly.
- Drive Command: "Feed to Position" (FP) (command word value of 0x10).
- Done output: when drive reaches "In Position"
- Type: Buffered

4.7.3 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Start	Input	BOOL	Start the Absolute Move
Position	Input	DINT	Position (motor/encoder steps, negative = reverse)
Speed	Input	REAL	Velocity of Move (rev/s)
Acc	Input	REAL	Acceleration of Move (rev/s/s)
Dec	Input	REAL	Deceleration (rev/s/s)
Sent	Output	BOOL	Move has been sent to drive
In_Progress	Output	BOOL	Move in progress
Done	Output	BOOL	Move completed without errors - desired position reached
Error	Output	BOOL	Move failed to complete - drive not ready or faulted
State	Output	DINT	Internal state of progress

4.8 AMP_Relative_Move Function Block

4.8.1 Overview

The Relative Move Function Block is used to move an axis by a specified distance (positive or negative) relative to its currently commanded position. The equivalent drive command is FL.

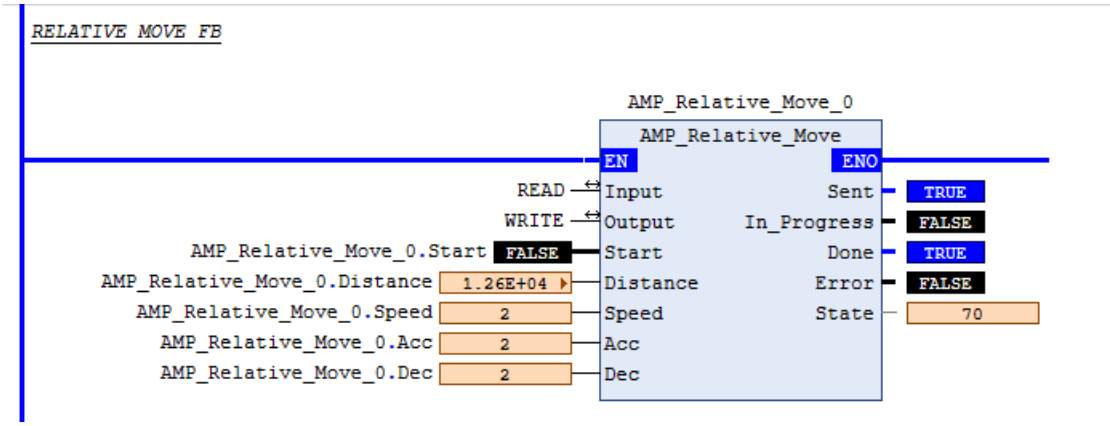


Figure 26: Applied Motion Relative Move Function Block

4.8.2 Functionality

The Relative Move uses a typical command sequence with the following specific differences:

- Parameters output to drive: relative Distance, as well as the correctly scaled values of parameters: Speed, Acc and Dec are sent via the output assembly.
- Drive Command: "Feed to Length" (FL) (command word value of 0x8).
- Done output: when drive reaches "In Position."
- Type: Buffered

4.8.3 Parameters:

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Start	Input	BOOL	Start the Relative Move
Distance	Input	DINT	Distance (motor/encoder steps, negative = reverse)
Speed	Input	REAL	Velocity of Move (rev/s)
Acc	Input	REAL	Acceleration of Move (rev/s/s)
Dec	Input	REAL	Deceleration (rev/s/s)
Sent	Output	BOOL	Move has been sent to drive
In_Progress	Output	BOOL	Move in progress
Done	Output	BOOL	Move completed without errors - desired position reached
Error	Output	BOOL	Move failed to complete - drive not ready or faulted
State	Output	DINT	Internal state of progress

4.9 AMP_Normal_Stop Function Block

4.9.1 Overview

The Normal Stop Function Block is used to stop and kill the drive using the normal deceleration set by the DE command. The equivalent drive command is SKD.

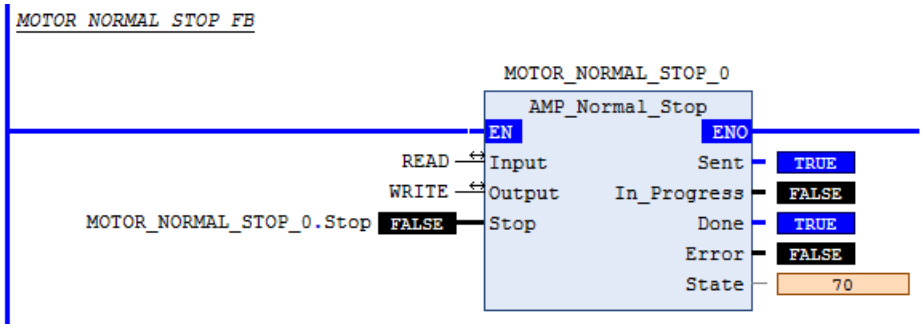


Figure 27: Applied Motion Normal Stop Function Block

4.9.2 Functionality

The Normal Stop Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: none.
- Drive Command: "Stop/Kill - DE " (command word value of 0x8000)
- Done output: when drive status indicates it is no longer “Moving”
- Type: Immediate

4.9.3 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Stop	Input	BOOL	Stop the Drive (on rising edge)
Sent	Output	BOOL	Stop has been sent to drive
In_Progress	Output	BOOL	Stop in progress
Done	Output	BOOL	Stop completed without errors – Not moving
Error	Output	BOOL	Stop failed to complete (Always false – Future use only)
State	Output	DINT	Internal state of progress

4.10 AMP_Crash_Stop Function Block

4.10.1 Overview

The Crash Stop Function Block is used to stop and kill the drive using the maximum deceleration as set by the AM command.

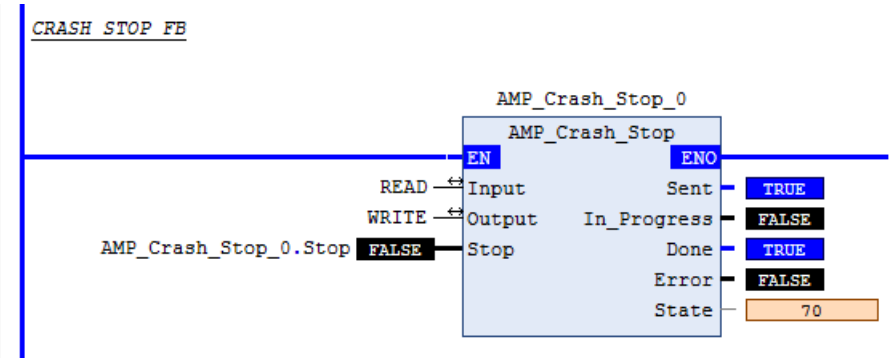


Figure 28: Applied Motion Crash Stop Function Block

4.10.2 Functionality

The Crash Stop Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: none.
- Drive Command: "Stop/Kill - AM" (command word value of 0x4000).
- Done output: when drive status indicates it is no longer “Moving”.
- Type: Immediate.

4.10.3 Parameters:

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Stop	Input	BOOL	Stop the Drive (on rising edge)
Sent	Output	BOOL	Stop has been sent to drive
In_Progress	Output	BOOL	Stop in progress
Done	Output	BOOL	Stop completed without errors – Not moving
Error	Output	BOOL	Stop failed to complete (Always false – Future use only)
State	Output	DINT	Internal state of progress

4.11 AMP_Alarm_Reset Function Block

4.11.1 Overview

The Alarm Reset Function Block is used to reset Drive Fault and clear Alarm Code (if possible). The equivalent drive command is AR.

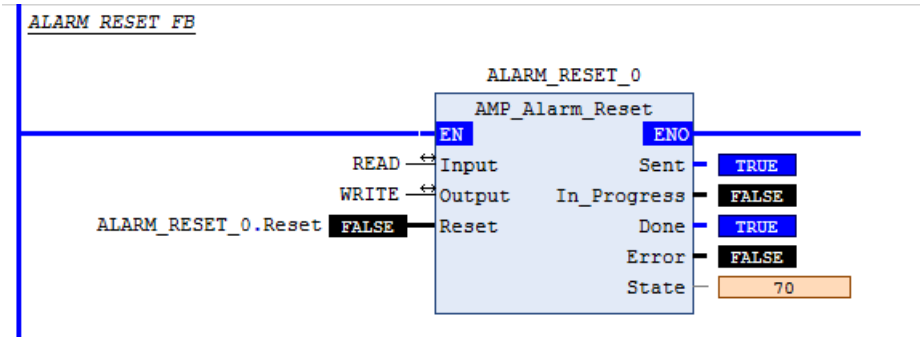


Figure 29: Applied Motion Alarm Reset Function Block

4.11.2 Functionality

The Alarm Reset Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: not changed.
- Drive Command: "Alarm Reset" (AR) (command word value of 0x100000).
- Done output: when drive no longer indicates “Drive Faulted” or “Alarm Present".
- Type: Immediate.

4.11.3 Parameters:

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Reset	Input	BOOL	Reset the Drive (on rising edge)
Sent	Output	BOOL	Reset has been sent to drive
In_Progress	Output	BOOL	Reset in progress
Done	Output	BOOL	Reset completed without errors – no alarm present
Error	Output	BOOL	Reset failed to complete (Always false – Future use only)
State	Output	DINT	Internal state of progress

4.12 AMP_SCL_Command_Execute Function Block

4.12.1 Overview

The SCL Command Execute Function Block executes selected SCL commands (as listed under “Available SCL Commands” in the Host Command Reference).

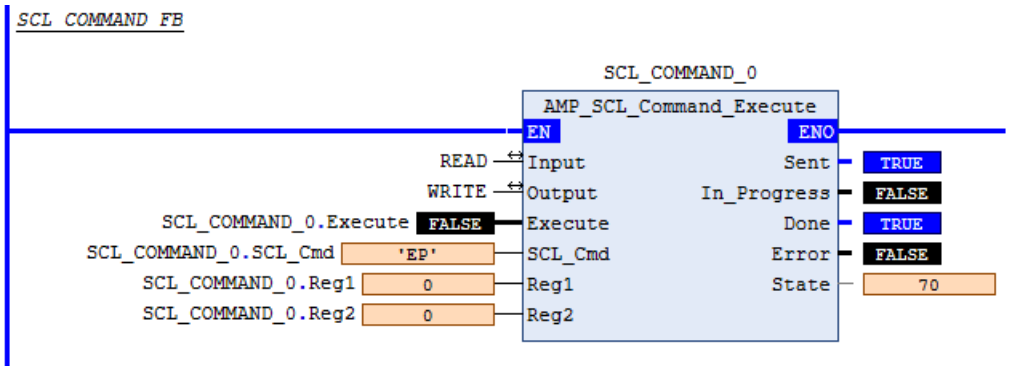


Figure 30: Applied Motion SCL Command Execute Function Block

4.12.2 Functionality

The SCL Command Execute Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: SCL Command sent as two ASCII characters into lower 16 bits of output assembly element 9. First character is copied into bits 8-15, second character is copied into bits 0-7. SCL Data/Registers Reg1 and Reg2 parameters are copied unchanged to output assembly elements 10 and 11.
- Drive Command: the command word value is set to 0x40000.
- Done output: when command is sent.
- Type: Immediate

4.12.3 Exceptions/Notes

The user can set Reg1 and/or Reg2 parameters to ASCII values when required by enclosing the character value in single quotes.

4.12.4 Parameters:

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Execute	Input	BOOL	Execute the SCL command
SCL_Command	Input	STRING(2)	SCL Command Characters (always 2)
Reg1	Input	DINT	Data / SCL Register 1*
Reg2	Input	DINT	Data / SCL Register 2*
Sent	Output	BOOL	Command has been sent to drive
In_Progress	Output	BOOL	In progress = sending command
Done	Output	BOOL	Command sent
Error	Output	BOOL	Command failed to be sent
State	Output	DINT	Internal state of progress

* Reg1 and/or Reg2 parameters can be set to ASCII values when required by enclosing the character value in single quotes.

4.13 AMP_Point_to_Point_IO Function Block

4.13.1 Overview

The Point-to-Point Move with I/O Function Block executes moves involving up to 2 Input conditions and a possible output state change. The move variant can be any of the following:

- FS - Feed to Sensor
- FD - Feed to Double Sensor
- FM - Feed to Sensor with Mask Distance
- FY - Feed to Sensor with Safety Distance
- FO - Feed to Length and Set Output
- FE - Follow Encoder
- SH - Seek Home

All seven (7) of the above variants need exactly one I/O point except for FD which requires a second I/O point to be specified in the parameters. The equivalent drive command is a combination of the chosen mnemonic among the 7 variants, along with either DC or VC.

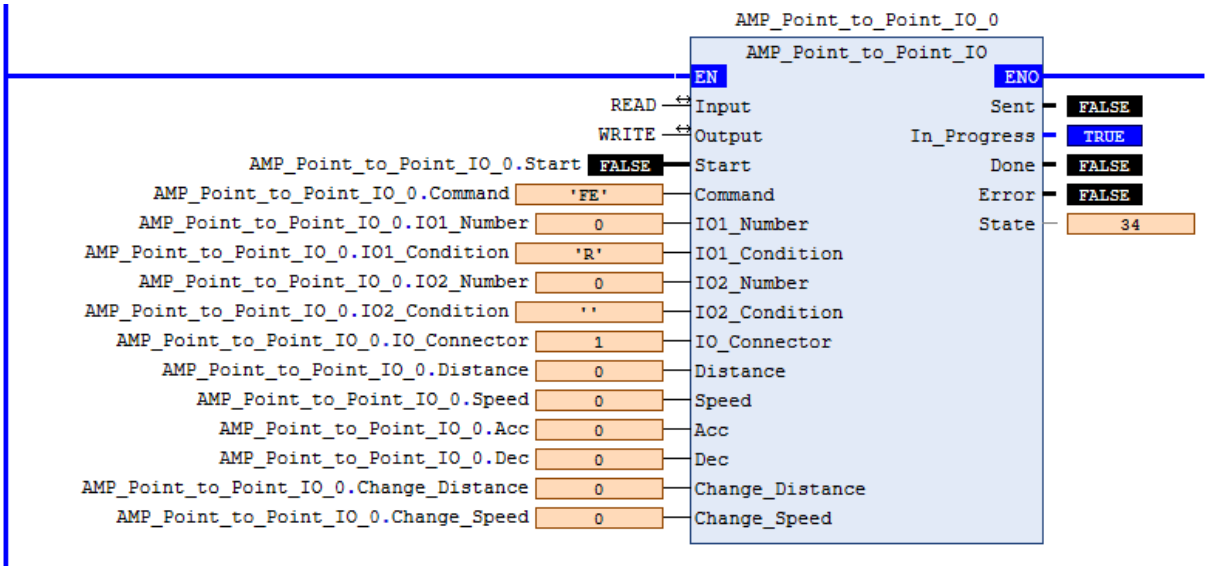


Figure 31: Applied Motion Point to Point with I/O Function Block

4.13.2 Functionality

The Point-to-Point Move with I/O uses a typical command sequence with the following specific differences:

- For the FD (Double Sensor) variation, the Change_Speed parameter is converted as required and sent using the VC SCL command (without requiring the user to perform this externally). This parameter is optional: it can be set to 0 to save execution time. In that case, the preset/previously set value will be used by the drive.
- For the FY, FM and FO variations, the Change_Distance parameter is converted as required and sent using the DC SCL command (without requiring the user to perform this externally). This parameter is optional: it can be set to 0 to save execution time. In that case, the preset/previously set value will be used by the drive.
- Parameters output to drive: Distance to move past the I/O detection, as well as the correctly scaled values of parameters: Speed, Acc and Dec are sent via the output assembly, Input/Output numbers, connector and conditions (FD requires both I/O points defined).
- Drive Command: depending on value of Command parameter (FS/FD/FY/FM/FO/SH/FE), the command word value is correspondingly set to one of 0x20, 0x40, 0x80, 0x100, 0x200, 0x800 or 0x2000.
- Done output: when drive reaches "In Position"
- Type: Buffered

4.13.3 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Start	Input	BOOL	Start the Move
Command	Input	STRING (2)	Command variant: FS/FD/FY/FM/FO/SH/FE
IO1_Number	Input	DINT	Main Input/Output Number (0 = Encoder index, 1-8 = In/Out 1-8)
IO1_Condition	Input	STRING (1)	Main Input/Output Condition/State H/L/R/F (High/Low/Rising Edge/Falling Edge)

IO2_Number	Input	DINT	Second Input/Output Number (0 = Encoder index, 1-8 = In/Out 1-8)
IO2_Condition	Input	STRING (1)	Second Input/Output Condition/State H/L/R/F (High/Low/Rising Edge/Falling Edge)
IO_Connector	Input	DINT	I/O Connector (1=IN/OUT1, 2=IN/OUT2)
Distance	Input	DINT	Distance to Move (negative = reverse)
Speed	Input	REAL	Velocity of Move (rev/s)
Acc	Input	REAL	Acceleration of Move (rev/s/s)
Dec	Input	REAL	Deceleration (rev/s/s)

Name	**Usage**	**Data Type**	**Style**	**Description**
Change_Distance	Input	DINT	Decimal	Change Distance (FY/FM/FO only - 0=Use existing DC set value)
Change_Speed	Input	REAL	Float	Velocity of Move after Change (rev/s – FD only -). 0=Use existing VC set value)
Sent	Output	BOOL	Decimal	Move has been sent to drive
In_Progress	Output	BOOL	Decimal	Move in progress
Done	Output	BOOL	Decimal	Move completed without errors - desired position reached
Error	Output	BOOL	Decimal	Move failed to complete - drive not ready or faulted
State	Output	DINT	Decimal	Internal state of progress

4.14 AMP_Point_to_Point_FC Function Block

4.14.1 Overview

The Point-to-Point Move with Speed Change Function Block executes a feed to length (relative move) with a speed change after an initial travel distance (FC command). The equivalent drive commands are DC, VC and FC.

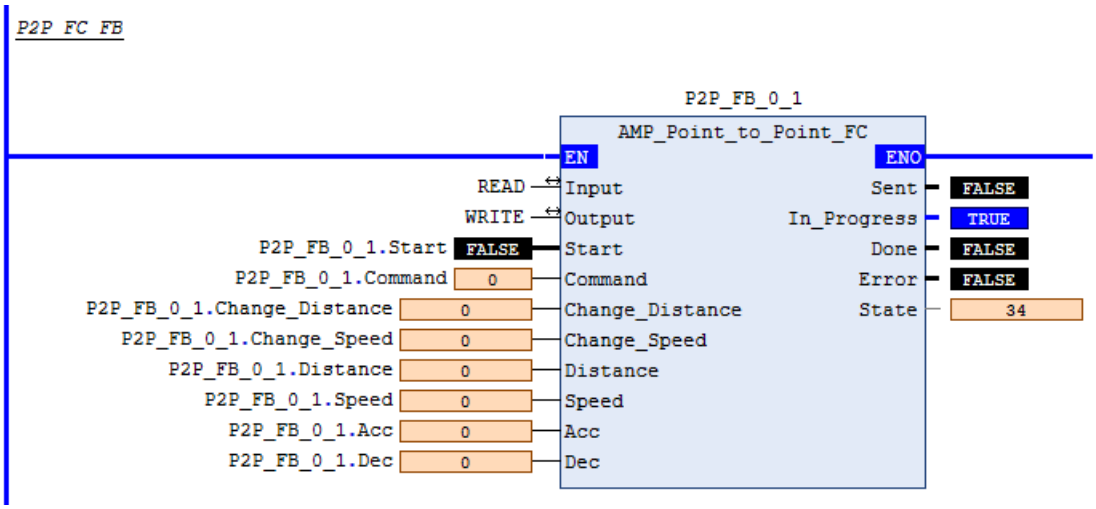


Figure 32: Applied Motion Point to Point with Speed Change Function Block

4.14.2 Functionality

The Point-to-Point Move with Speed Change uses a typical command sequence with the following specific differences:

- Change_Distance and Change_Speed parameters are converted as required and sent using SCL (without requiring the user to perform this externally). These parameters are optional: they can be set to 0 to save execution time. In that case, the preset/previously set values will be used by the drive.
- Parameters output to drive: Total distance to move, as well as the correctly scaled values of parameters: Speed, Acc and Dec are sent via the output assembly.
- Drive Command: FC, the command word value is set to 0x400.
- Done output: when drive reaches "In Position".
- Type: Buffered

4.14.3 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Start	Input	BOOL	Start the Move
Change_Distance	Input	DINT	Distance to Move before Speed Change (negative = reverse). (0=Use existing DC set value)
Change_Speed	Input	REAL	Velocity of Move after Change (rev/s). (0=Use existing VC set value)
Distance	Input	DINT	Overall Distance to Move (negative = reverse)
Speed	Input	REAL	Velocity of Move before Change (rev/s)
Acc	Input	REAL	Acceleration of Move (rev/s/s)
Dec	Input	REAL	Deceleration (rev/s/s)
Sent	Output	BOOL	Move has been sent to drive
In_Progress	Output	BOOL	Move in progress
Done	Output	BOOL	Move completed without errors - desired position reached
Error	Output	BOOL	Move failed to complete - drive not ready or faulted
State	Output	DINT	Internal state of progress

4.15 AMP_Jog_Move Function Block

4.15.1 Overview

The Jog Move Function Block is used to start jogging an axis at a specified speed and direction (a negative speed set point implies a CCW direction). The equivalent drive commands are JA, JL, JS and CJ.

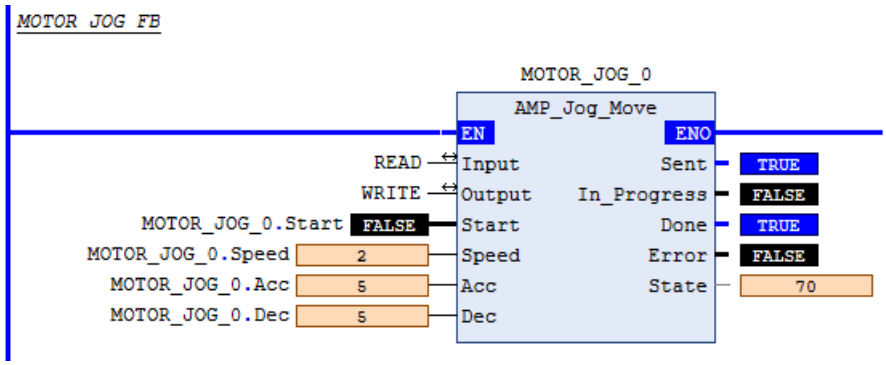


Figure 33: Applied Motion Jog Move Function Block

4.15.2 Functionality

The Jog Move Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: The correctly scaled values of parameters: Speed, Acc and Dec are written to the output assembly elements 1, 2 and 3, respectively.
- Drive Command: "Start Jogging" (command word value of 0x10000).
- Done output: when drive status indicates it is “Moving”.
- Type: Buffered

4.15.3 Exceptions/Notes

The Jog Move only uses the rising edge of the Start parameter to start the jogging. To stop the motion, the user will need to use either the AMP_Normal_Stop or the AMP_Crash_Stop Function Block. Alternately, you may use AMP_Update_Jog_Speed to set the speed to 0, this will indeed cause the motor to stop, but the drive will still report that it is jogging – this may cause confusion in some cases.

4.15.4 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Start	Input	BOOL	Start Jogging*
Speed	Input	REAL	Velocity of Move (rev/s) (negative = CCW)
Acc	Input	REAL	Acceleration of Move (rev/s/s)
Dec	Input	REAL	Deceleration (rev/s/s)
Sent	Output	BOOL	Move has been sent to drive
In_Progress	Output	BOOL	Move in progress
Done	Output	BOOL	Move completed without errors - desired position reached
Error	Output	BOOL	Move failed to complete - drive not ready or faulted
State	Output	DINT	Internal state of progress

*The Jog Move Function Block uses the rising edge of the Start parameter to start the jogging. To stop the motion, use either the AMP_Normal_Stop Function Block or the AMP_Crash_Stop Function Block.

4.16 AMP_Update_Jog_Speed Function Block

4.16.1 Overview

The Update Jog Speed Function Block is used to modify the speed of an ongoing jog motion. The equivalent drive command is CS.

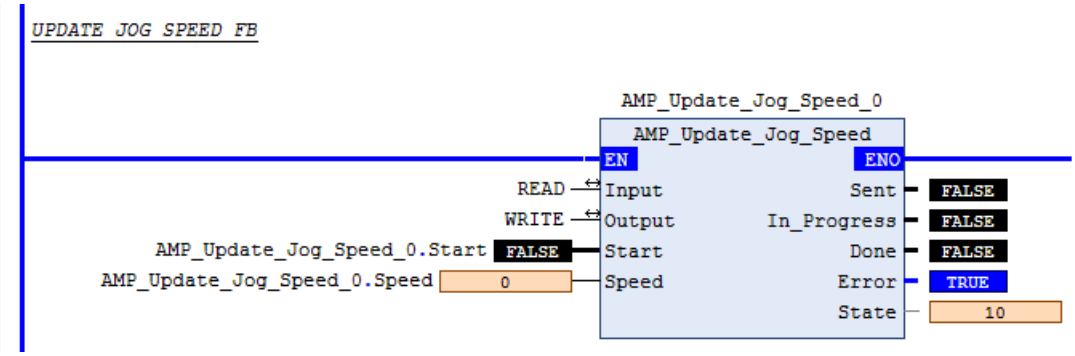


Figure 34: Applied Motion Update Jog Speed Function Block

4.16.2 Functionality

The Update Jog Speed Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: The correctly scaled values of parameter “Speed” is written to the output assembly element 1.
- Drive Command: "Update Jog Speed" (command word value of 0x20000).
- Done output: immediately when command is sent.
- Type: Immediate

4.16.3 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Update	Input	BOOL	Update the Jogging Speed
Speed	Input	REAL	Updated Jog Speed (rev/s), (negative = CCW)
Sent	Output	BOOL	Update has been sent to drive
In_Progress	Output	BOOL	Update in progress
Done	Output	BOOL	Update completed without errors
Error	Output	BOOL	Update failed to complete - drive not ready or faulted
State	Output	DINT	Internal state of progress

4.17 AMP_Q_Segment_Execute Function Block

4.17.1 Overview

The Queue Segment Execute Function Block is used to load a specified segment into the queue and execute it. The equivalent drive command is QX.

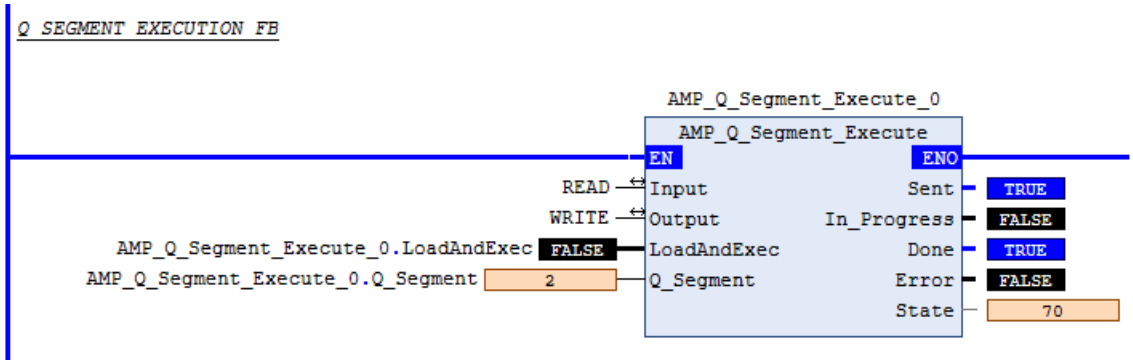


Figure 35: Applied Motion Q Segment Launch and Execute Function Block

4.17.2 Functionality

The Queue Segment Execute Function Block uses a typical command sequence with the following specific differences:

- Parameters output to drive: The Q_Segment parameters is copied to output assembly element 10.
- Drive Command: "Q Load and Execute (QX)", command word value of 0x80000.
- Done output: immediately when command is sent.
- Type: Buffered

4.17.3 Parameters:

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
LoadAndExec	Input	BOOL	Load and Execute the provided Q segment
Q_segment	Input	DINT	Q segment number to load and execute
Sent	Output	BOOL	Q Load and Execute has been sent to drive
In_Progress	Output	BOOL	Q Load and Execute in progress
Done	Output	BOOL	Q Load and Execute completed without errors
Error	Output	BOOL	Q Load and Execute failed to complete - drive not ready or faulted
State	Output	DINT	Internal state of progress

4.18 AMP_Hard_Stop_Homing Function Block

4.18.1 Overview

The Hard Stop Homing Function Block accepts all the required parameters for SCL commands HV1, HV2, HV3, HA1, HA2, HA3, HL1, HL2, HL3, HC, HD, HO and then sends the HS
command to initiate the Hard Stop Homing move.

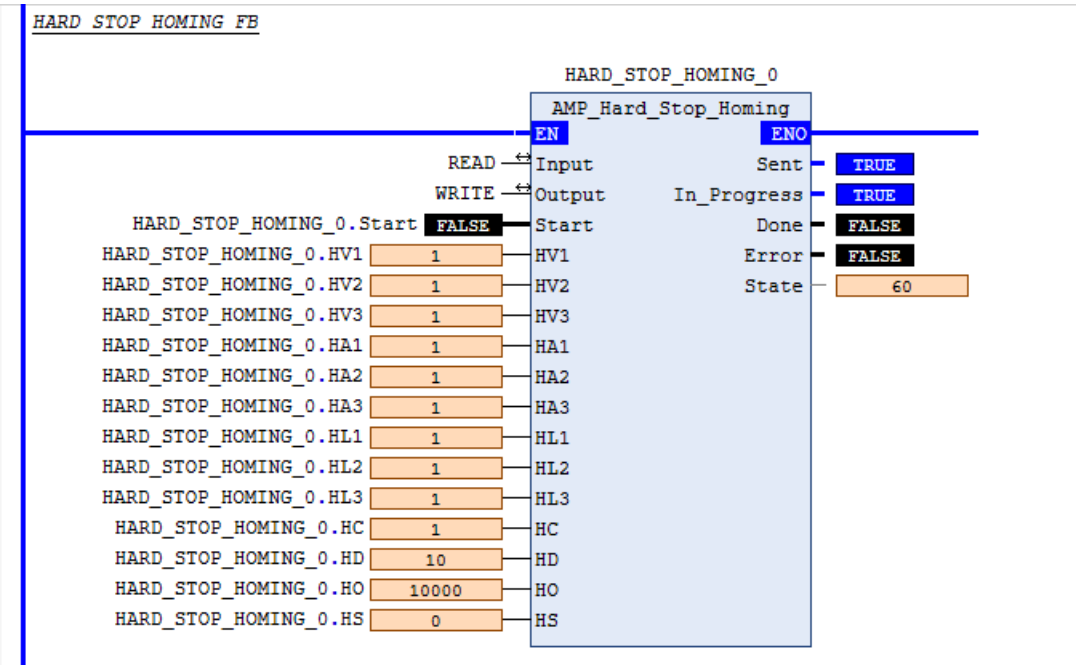


Figure 36: Applied Motion Hard Stop Homing Function Block

4.18.2 Functionality

The Hard Stop Homing Function Block uses a typical command sequence with the following specific differences:

- The command sequence is repeated for each of HV1, HV2, HV3, HA1, HA2, HA3, HL1, HL2, HL3, HC, HD, HO, HS in order to set parameters and finally start the homing.
- The SCL Command letters (output element 9) are set for the corresponding parameter.
- The Data/SCL Register 1 (output element 10) is set to the ASCII values of ‘1’, ‘2’ or ‘3’ for commands that require this, otherwise it is set to the correctly scaled value supplied as the corresponding parameter.
- The Data/SCL Register 2 (output element 11) is set as required to the correctly scaled value of the corresponding parameter.
- Drive Command: “Send Host Command”, i.e. the command word value is set to 0x40000. (The value will return to 0=idle between successive SCL command execution cycles).
- Done output: when drive reaches "In Position".
- Sent output: when the HS command has been sent.
- Type: Buffered

4.18.3 Parameters

Name	Usage	Data Type	Description
Input	InOut	ARRAY [0..13] OF DINT	Drive Input Assembly
Output	InOut	ARRAY [0..15] OF DINT	Drive Output Assembly
Start	Input	BOOL	Start the Homing
HV1	Input	REAL	Velocity used while seeking the hard stop (rev/s)
HV2	Input	REAL	Velocity when moving offset distance set by HO (rev/s)
HV3	Input	REAL	Velocity when searching for encoder index if desired (rev/s)
HA1	Input	REAL	Accel used while seeking the hard stop (rev/s/s)
HA2	Input	REAL	Accel when moving offset distance set by HO (rev/s/s)
HA3	Input	REAL	Accel when searching for encoder index if desired (rev/s/s)
HL1	Input	REAL	Decel used while seeking the hard stop (rev/s/s)
HL2	Input	REAL	Decel when moving offset distance set by HO (rev/s/s)
HL3	Input	REAL	Decel when searching for encoder index if desired (rev/s/s)
HC	Input	REAL	Hard Stop Current (Amps)
HD	Input	DINT	Hard Stop Fault Delay (ms)
HO	Input	DINT	Home Offset (steps, negative = CCW, positive = CW)
HS	Input	DINT	Hard Stop method (0 = without index, 1 = with index)
Sent	Output	BOOL	Command has been Sent to drive
In_Progress	Output	BOOL	Command in progress
Done	Output	BOOL	Command completed without Errors
Error	Output	BOOL	Command failed
State	Output	DINT	Internal state of progress